

NAME: Ganesh Balkrushna Sonawane

Roll no: 43

Create a Student Record Management System using linked list

- Use a singly/doubly linked list to store student data (Roll No, Name, Marks).
- Perform operations: Add, Delete, Update, Search, and Sort.
- Display records in ascending/descending order based on marks or roll number.

'''

```
class StudentNode:
```

```
    def __init__(self, roll_no, name, marks):
```

```
        self.roll_no = roll_no
```

```
        self.name = name
```

```
        self.marks = marks
```

```
        self.next = None
```

```
class StudentLinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def add_student(self, roll_no, name, marks):
```

```
        new_node = StudentNode(roll_no, name, marks)
```

```
        if self.head is None:
```

```
            self.head = new_node
```

```
        else:
```

```
            current = self.head
```

```
            while current.next:
```

```
        current = current.next

    current.next = new_node

    print(" Student record added.")
```

```
def delete_student(self, roll_no):

    current = self.head

    prev = None

    while current:

        if current.roll_no == roll_no:

            if prev:

                prev.next = current.next

            else:

                self.head = current.next

            print("Student record deleted.")

            return

        prev = current

        current = current.next

    print("Student not found.")
```

```
def update_student(self, roll_no, new_name, new_marks):

    current = self.head

    while current:

        if current.roll_no == roll_no:

            current.name = new_name

            current.marks = new_marks

            print(" Student record updated.")
```

```
        return

    current = current.next

    print(" Student not found.")
```

```
def search_student(self, roll_no):

    current = self.head

    while current:

        if current.roll_no == roll_no:

            print(f"Found: Roll No: {current.roll_no}, Name: {current.name}, Marks: {current.marks}")

            return

        current = current.next

    print(" Student not found.")
```

```
def display_students(self, sort_by="roll_no", ascending=True):

    students = []

    current = self.head

    while current:

        students.append((current.roll_no, current.name, current.marks))

        current = current.next

    if sort_by == "roll_no":

        students.sort(key=lambda x: x[0], reverse=not ascending)

    elif sort_by == "marks":

        students.sort(key=lambda x: x[2], reverse=not ascending)
```

```
if not students:
```

```
    print(" No records to display.")
```

```
    return
```

```
print("\n Student Records:")
```

```
for s in students:
```

```
    print(f"Roll No: {s[0]}, Name: {s[1]}, Marks: {s[2]}")
```

```
def menu():
```

```
    system = StudentLinkedList()
```

```
    while True:
```

```
        print("\n--- Student Record Management Menu ---")
```

```
        print("1. Add Student")
```

```
        print("2. Delete Student")
```

```
        print("3. Update Student")
```

```
        print("4. Search Student")
```

```
        print("5. Display All Students")
```

```
        print("6. Exit")
```

```
choice = input("Enter your choice (1-6): ")
```

```
if choice == '1':
```

```
    roll = int(input("Enter Roll No: "))
```

```
    name = input("Enter Name: ")
```

```
    marks = int(input("Enter Marks: "))
```

```
    system.add_student(roll, name, marks)

elif choice == '2':

    roll = int(input("Enter Roll No to Delete: "))

    system.delete_student(roll)

elif choice == '3':

    roll = int(input("Enter Roll No to Update: "))

    name = input("Enter New Name: ")

    marks = int(input("Enter New Marks: "))

    system.update_student(roll, name, marks)

elif choice == '4':

    roll = int(input("Enter Roll No to Search: "))

    system.search_student(roll)

elif choice == '5':

    sort_key = input("Sort by 'roll_no' or 'marks': ").strip()

    order = input("Order 'asc' or 'desc': ").strip()

    ascending = True if order == 'asc' else False

    system.display_students(sort_by=sort_key, ascending=ascending)

elif choice == '6':

    print(" Exiting Student Record Management System.")

    break

else:

    print("Invalid choice. Try again.")
```

```
menu()
```

""OUTPUT

gescoe@gescoe-OptiPlex-3020:~/SE B \$ python3 linklist.py

--- Student Record Management Menu ---

1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. Display All Students
6. Exit

Enter your choice (1-6): 1

Enter Roll No: 5

Enter Name: hii

Enter Marks: 45

Student record added.

--- Student Record Management Menu ---

1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. Display All Students
6. Exit

Enter your choice (1-6): 2

Enter Roll No to Delete: 3

Student not found.

--- Student Record Management Menu ---

1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. Display All Students
6. Exit

Enter your choice (1-6): 3

Enter Roll No to Update: 6

Enter New Name: hello

Enter New Marks: 56

Student not found.

--- Student Record Management Menu ---

1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. Display All Students
6. Exit

Enter your choice (1-6): 4

Enter Roll No to Search: 5

Found: Roll No: 5, Name: hii, Marks: 45

--- Student Record Management Menu ---

1. Add Student

2. Delete Student
3. Update Student
4. Search Student
5. Display All Students
6. Exit

Enter your choice (1-6): 5

Sort by 'roll\_no' or 'marks': marks

Order 'asc' or 'desc': desc

Student Records:

Roll No: 5, Name: hij, Marks: 45

--- Student Record Management Menu ---

1. Add Student
2. Delete Student
3. Update Student
4. Search Student
5. Display All Students
6. Exit

Enter your choice (1-6): 6

Exiting Student Record Management System.

""